

03

| SECTION 03

AI AGENT

아키텍처 기초

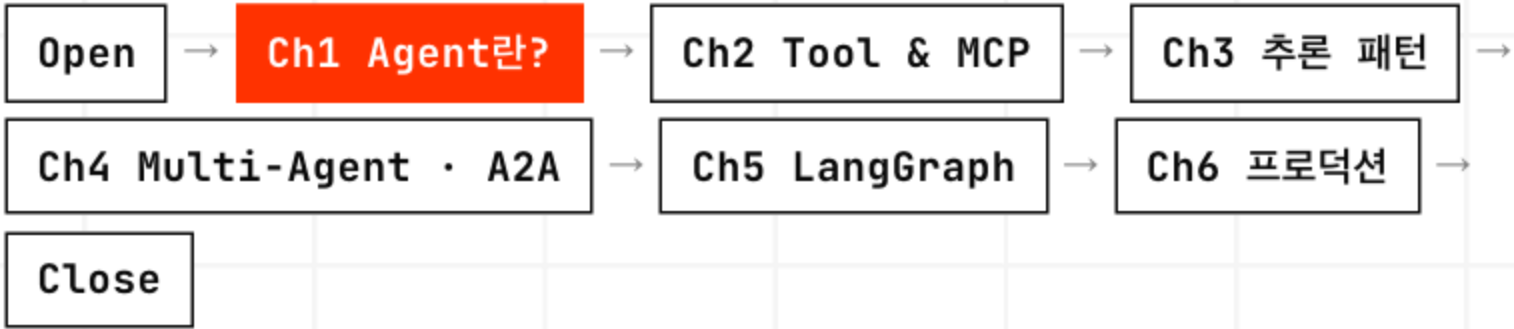
Loop + Tools + State + Policy

AI SERVICE ARCHITECT / 2026 / 1H 30M

— LEARNING GOALS

오늘의 목표

- 단순 LLM 호출과 Agent의 차이
- ReAct · ReWOO · Plan-and-Act 추론 패턴 진화
- Tool Use & MCP · A2A · ACP 표준 프로토콜
- Multi-Agent 시스템 설계 기준
- StateGraph + Checkpointer로 기본 Agent 구현



CHAPTER 1 / 6

01 / 6

AGENT란 무엇인가?

이 챕터에서 답할 질문

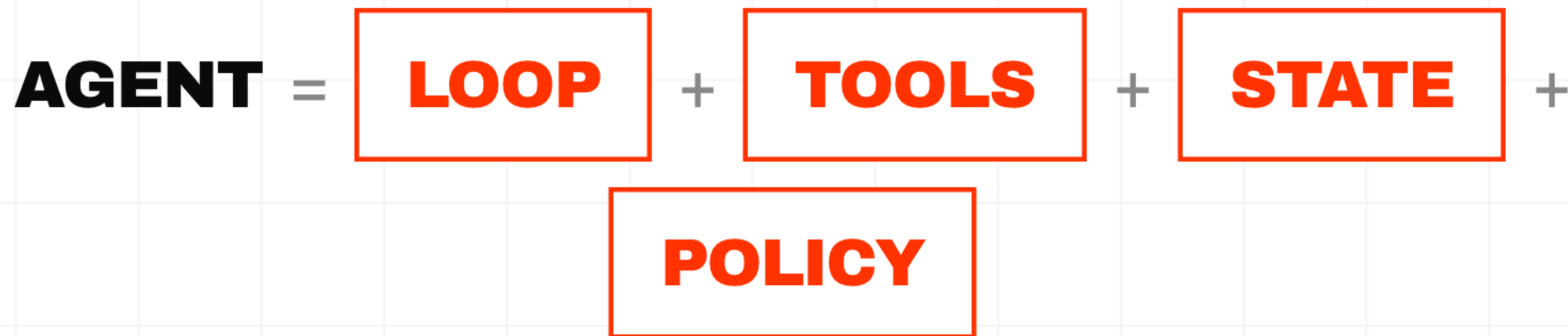
- 단순 LLM 호출과 Agent의 차이는 무엇인가?
- **Agent = Loop + Tools + State + Policy** 공식의 의미
- Agent가 만들어내는 5가지 새로운 문제는?
- 그 문제가 실제 사고로 어떻게 번지는가?

— WHAT IS AN AGENT?

챗봇과 AGENT는 무엇이 다른가

	LLM 호출	AGENT
입출력	1-shot	루프(loop)
외부 세계	없음	도구(Tool Use)
상태	대화 맥락뿐	명시적 State
결정	유저 지시	자가 결정

— THE FORMULA



LOOP 여러 단계	TOOLS 외부 연결	STATE 진행 상황	POLICY 가드레일
----------------------	-----------------------	-----------------------	-----------------------

네 요소 중 하나라도 빠지면 Agent가 아님.

— NEW PROBLEMS

AGENT가 만드는 새 문제 5가지

01**비결정성**

같은 입력이어도 루프 경로
가 다름 · 테스트 어려움

02**FAN-OUT 비용**

한 요청당 LLM 여러 번 →
비용·지연 곱해짐

03**무한루프 위험**

step 제한·토큰 예산 필수

04**실패 복구**

툴 실패 시 어디서 재개?

05**권한·안전**

도구가 실 시스템을 건드릴
→ 감사 필수

— INCIDENT · FIVE FAILURES

AGENT 5 문제 실제 사고 사례

01 비결정성 어제 통과한 경로가 오늘 달라짐 → QA '버그' vs 개발 '재현 안 됨'	02 FAN-OUT 비용 한 질문에 LLM 17번 · 1인당 \$0.85 → 무료 사용자 1K = 월 \$25K	03 무한루프 Tool 실패 시 '재시도' LLM 판단 → 8시간 만에 토큰 예산 고갈	04 실패 복구 항공권 결제 후 호텔 예약 실패 → 반쯤 결제된 어정쩡한 상태	05 권한·안전 "내 데이터 다 지워줘" → LLM이 delete 도구 실행, 승인 없음
--	--	--	---	---

전부 앱 서비스엔 없던 실패 양상. Agent 아키텍트는 안정성 체크리스트의 재설계 필요.

CHAPTER 2 / 6

02 / 6

TOOL USE & MCP

이 챕터에서 답할 질문

- **Function Calling**은 어떻게 동작하는가?
- Tool Use 아키텍처 **4 요소**는 무엇인가?
- **MCP**는 왜 등장했고 무엇을 표준화했는가?
- MCP 생태계 (Registry · 75+ 커넥터) **활용 전략**

TOOL USE = FUNCTION CALLING



모든 주요 Provider 표준 지원. Tool Use는 옵션이 아니라 Agent의 기본 근육.

— ARCHITECTURE ELEMENTS

TOOL USE 아키텍처 4 요소

01 TOOL REGISTRY 이름 · 스키마 · 설명 · 권한 tag	02 TOOL EXECUTOR 샌드박스 · 타임아웃 · 리소스 제한	03 RESULT FORMATTER 긴 결과 요약 · PII 제거	04 POLICY LAYER 누가 언제 어떤 도구를 쓰는가
--	--	---	---

TOOL REGISTRY — 무엇을 저장하나

저장 항목

- **name** — LLM이 호출하는 식별자
- **description** — 언제 쓰는지 자연어 설명
- **input_schema** — JSON Schema · 필수/선택
- **output_schema** — 반환 타입 (선택)
- **version** — semver · 호환성 관리
- **owner / team** — 책임 주체 추적
- **permission tag** — read · write · payment

예시 (JSON SCHEMA)

```
{
  "name": "get_weather",
  "description": "도시의 실시간 날씨를 반환",
  "input_schema": {
    "type": "object",
    "properties": {"city": {"type": "string"}},
    "required": ["city"]
  },
  "version": "1.2.0",
  "owner": "platform-team",
  "tags": ["read", "external_api"]
}
```

중앙 관리해야 도구 발견 · 거버넌스 · 버전 호환성 가능 · 흩어두면 추적 불가.

TOOL EXECUTOR — AGENT에 결과를 전달하는 흐름

한 호출 사이클

- 1. LLM — tool_call JSON 발신 (name + args + tool_use_id)
- 2. Executor — schema 검증 → sandbox 실행
- 3. External — 실제 함수 · API · DB · Shell 호출
- 4. Result — raw 반환 + 메타 (duration · status)
- 5. tool_result — tool_use_id 매칭으로 Agent State에 주입

AGENT 결과 전달 구조

- messages 배열에 {role: "tool", tool_use_id, content} append
- 다음 LLM 호출에 자동 포함 → 모델이 결과 보고 다음 판단

EXECUTOR의 안전 장치

- Sandbox — subprocess · container · WASM 격리
- Timeout — 호출당 상한, 무한 대기 차단
- Retry · Idempotency 키 — 멍등성 보장 후 재시도
- 자원 제한 — mem · cpu · net 쿼터
- 관측 — 호출당 trace · cost 태깅

실패 처리 원칙

- 예외 throw 금지 — 에러도 tool_result로 반환
- 모델이 에러 메시지 보고 스스로 복구 시도

tool_use_id로 요청·응답 매칭 · 결과는 tool_result 메시지로 Agent 컨텍스트에 주입.

RESULT FORMATTER — LLM에 들어가기 전 정제

왜 필요한가

- raw 결과 그대로 넣으면 **컨텍스트 윈도우 폭발** (10MB JSON · 1만 row)
- PII · 시크릿이 LLM 메시지에 그대로 노출
- HTML · binary · 비표준 포맷은 LLM이 잘못 해석

처리 동작

- **truncate / summarize** — top-N · 요약
- **redact** — PII · 토큰 · 키 마스킹
- **convert** — HTML → markdown · binary → 메타데이터

BEFORE / AFTER 예시

BEFORE

```
[10,234 rows]
{"tx_id":"a3f...", "amount":1029.5,
"card":"4532-1234-5678-9012",
"user_email":"alice@x.com", ...}
[10,234개 더 ...]
```

AFTER

```
Top 5 by amount:
1. tx_a3f - $1,029.50 (card: ****9012)
2. tx_b71 - $890.30 (card: ****4421)
...
총 10,234건 · 합 $891,203
```

"raw → formatted"는 **비용·보안 게이트** · context 잠식과 PII 누출을 동시에 막음.

POLICY LAYER — 누가 · 언제 · 어떤 도구

입력

- user / role — 호출 주체
- tool name + permission tag
- arguments — 금액 · 대상 · 환경
- context — 이전 호출 · 시간대 · 세션

결정

- allow · 그대로 실행
- deny · 차단 + 사유
- require_approval · HITL interrupt

구현 패턴

- 룰 엔진 · OPA (Open Policy Agent)
- Tool에 **permission tag**로 라벨링
- 모든 결정은 **감사 로그**에 기록

현실적 규칙 예시

- payment.* → 관리자 + HITL 필수
- delete.* → 프로덕션 차단
- email.send → 내부 도메인만
- db.write → working hours만

Policy 없으면 LLM 오판 한 번이 곧 사고 · 사용자 농담을 진지하게 받아 데이터 삭제한 사례 다수.

— MANY TOOLS · ANTHROPIC 표준 해법

도구가 많아질 때 — TOOL SEARCH TOOL · PROGRAMMATIC CALLING

문제 MCP 서버 5개 · 도구 50+ 연결 시 정의만 ~77K 토큰이 컨텍스트를 잠식 (대화 전부터). GitHub MCP 하나로 Sonnet 컨텍스트 1/4 점유.

01

TOOL SEARCH TOOL

defer_loading: true로 마킹 · 필요할 때만 검색 후 로드 · 77K → 8.7K 토큰 (85% 절감) · Opus 4.5 정확도 79.5% → 88.1% · 2026.01 Claude Code 기본 탑재

02

PROGRAMMATIC TOOL CALLING

LLM이 Python 코드를 작성해 도구들을 오케스트레이션 · 결과는 sandbox에서 처리, 컨텍스트 진입은 필요한 것만 · 43K → 27K 토큰 (37% 절감)

03

TOOL-RAG 패턴

질문을 받고 관련 있을 만한 도구 5~10개만 동적 노출 · 도구도 RAG의 retrieval source처럼 취급 · 자체 구현도 가능, MCP Anthropic API가 1급 지원

좋은 description은 여전히 필수 — 검색·매칭의 입력은 결국 도구 description. 하지만 모두 등록 ≠ 모두 노출.

— MODEL CONTEXT PROTOCOL

MCP · LLM 앱과 도구의 USB-C

등장 배경

- 모든 Agent 팀이 SaaS·DB 통합을 자체 구현
- Function Calling은 LLM ↔ Tool만 표준화
- "도구 자체"의 표준 인터페이스 부재

MCP가 표준화한 4가지

- Tools — 호출 가능한 함수
- Resources — 읽기 전용 데이터
- Prompts — 재사용 템플릿
- Sampling — Server가 Client LLM 호출 (역방향)

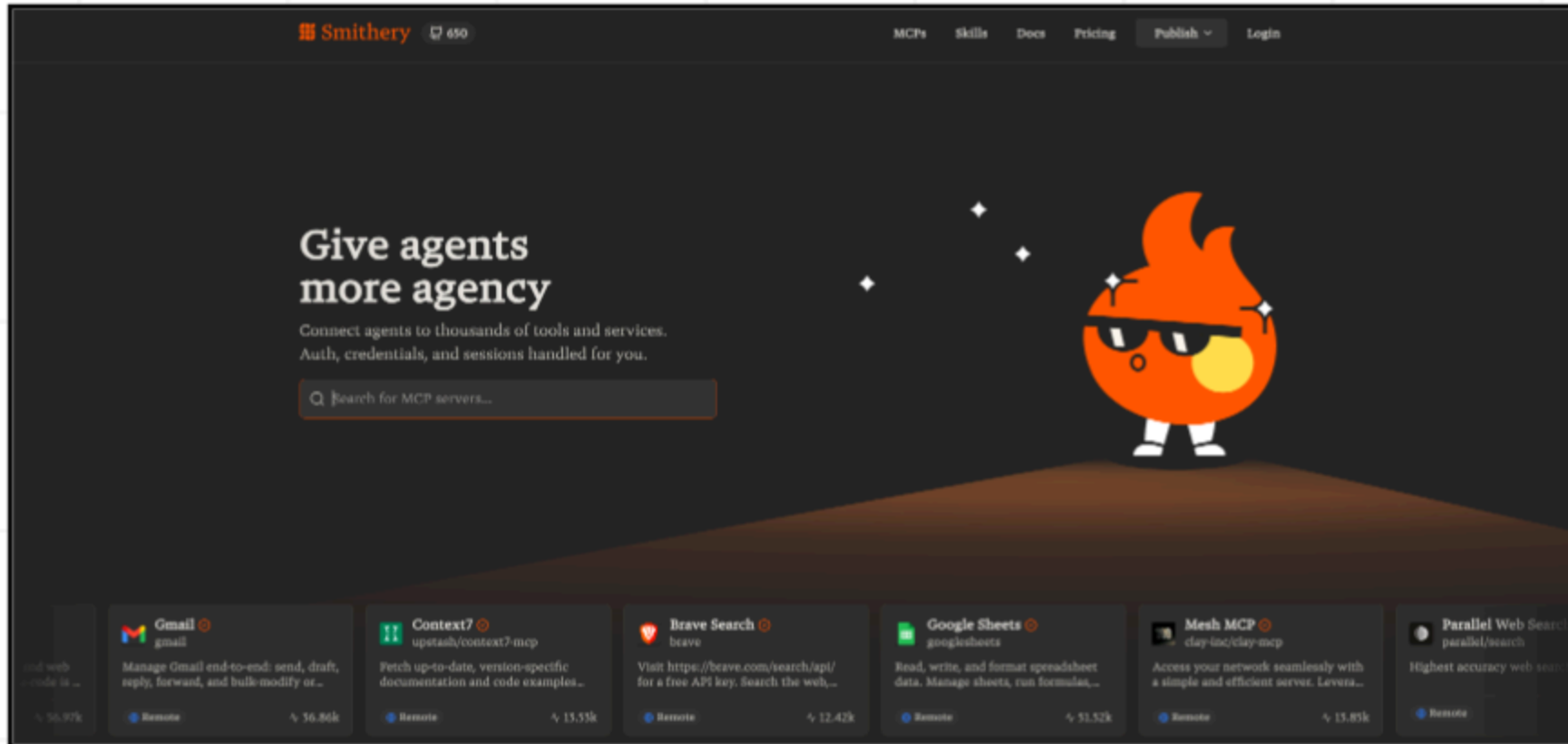
2026 거버넌스

- 2024-11 Anthropic 공개
- 2025-11 사양 갱신
(Async · Stateless · Server identity)
- 2026 Linux Foundation Agentic AI Foundation 이관
- 후원: Anthropic · Block · OpenAI · Google · MS · AWS · Cloudflare · Bloomberg
- SDK 다운로드 9,700만+/월
- Registry 검증 커넥터 75+

1년 뒤엔 자체 인터페이스가 부채 — 표준에 맞추라는 신호.

— MARKETPLACE · SMITHERY

MCP MARKETPLACE · 검증된 SERVER를 즉시 연결



smithery.ai · 650+ MCP Server · OAuth · 원클릭 연결

MARKETPLACE 역할

- **Server 카탈로그** — Gmail · Slack · GitHub · Notion · Sheets 등 SaaS 통합 즉시 사용
- **Auth · Credentials 위임** — OAuth · API Key · Session을 카탈로그가 대신 처리
- **Hosted Remote** — Server를 카탈로그가 호스팅, Client는 URL만 연결

대표 카탈로그

- **Smithery** — 650+ Server · 검색 · 통계 · OAuth 자동
- **MCP Registry (공식)** — 사양 기반 Server 메타데이터
- **Anthropic Connector Directory** — Claude 공식 큐레이션

도구 통합이 NPM 패키지 설치 수준으로 내려옴 · SaaS는 Marketplace, 사내·규제는 자체 Server.

CHAPTER 3 / 6

03 / 6

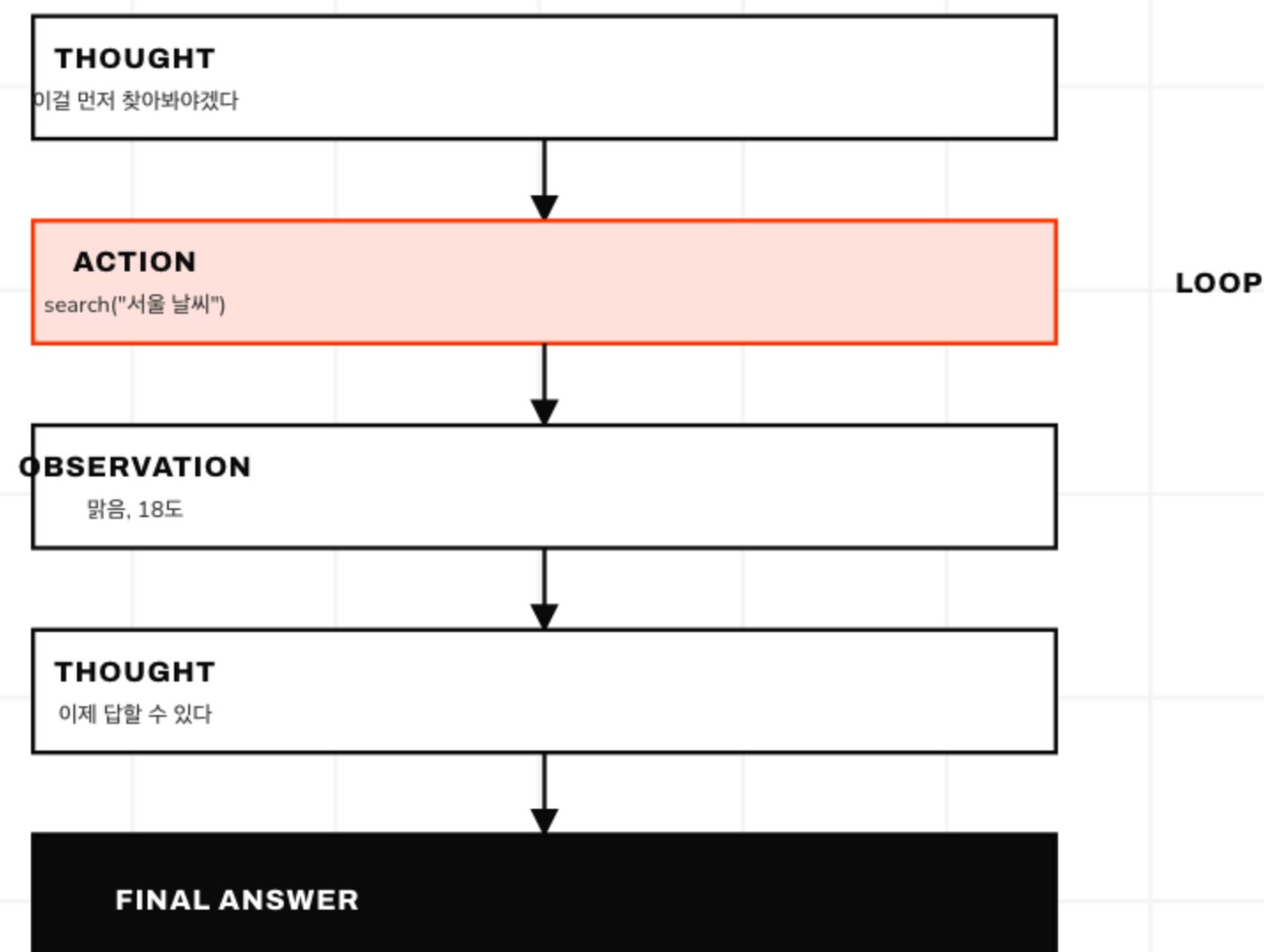
AGENT 추론 패턴

이 챕터에서 답할 질문

- ReAct 패턴의 본질과 한계는 무엇인가?
- ReWOO는 ReAct의 어떤 비효율을 해결하는가?
- Plan-and-Act는 장기 작업에서 왜 강한가?
- ReAct · ReWOO · Plan-and-Act는 어떤 작업에 맞는가?

REACT 패턴

Yao et al., 2022 · ICLR 2023



CoT의 환각을 실제 툴 관찰(Observation)이 현실 데이터로 교정.

— IMPACT & LIMITS

REACT의 효과와 한계

효과

- HotpotQA · FEVER — 환각 감소
- WebShop — Imitation·RL 대비 **+10%p**
- ALFWorld — **+34%p**
- in-context 예시 1~2개로 달성

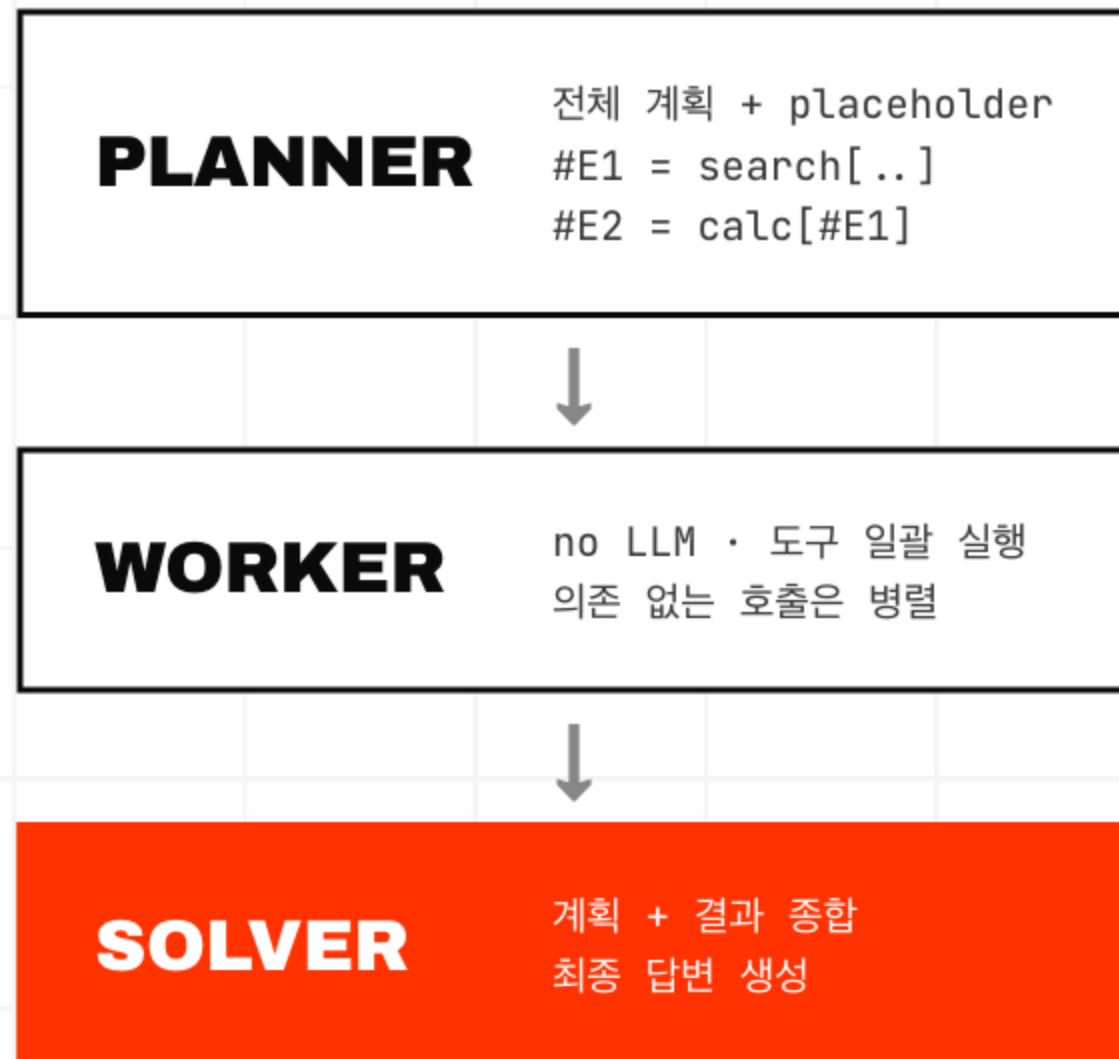
한계

- 작은 모델은 포맷 깨짐
- 2026: 14B급이 75%+ 안정성
- 실전에선 **Function Calling**으로 구조화 권장

→ [arXiv:2210.03629](#) · [Google Research Blog](#)

REWOO · 계획·실행 분리

Xu et al., 2023 · NeurIPS



왜 REACT보다 빠른가

- ReAct: 매 단계마다 LLM 호출
- ReWOO: LLM 호출 = 2번 (Plan + Solve)
- 도구 호출은 의존 따라 병렬

효과 (HOTPOTQA)

- 토큰 5배 절감
- 정확도 +4%
- 지연 50~70% 감소

한계

- 계획 오류 시 중간 교정 어려움
- 호출 패턴 예측 가능한 도메인에 강함

PLAN-AND-ACT · 장기 작업 전용

UC Berkeley · ICML 2025 (arXiv:2503.09572)

PLANNER

고수준 계획 생성

1. 검색 진입 → 2. 필터
→ 3. 정렬 → 4. 클릭 ...

큰 LLM

↓ 1단계 위임

EXECUTOR

click · type · scroll

환경 피드백 반영

작은 LLM

↻ Re-Plan?

핵심 통찰

- 장기 작업은 **계획 ≠ 실행**
- 한 모델로 둘 다 잘하기 어려움
- 역할별 모델 크기 분리

학습 데이터

- 실제 trajectory에서
합성 plan 역생성
- Planner 일반화 능력 향상

결과

- WebArena-Lite **57.6% SOTA**
- WebVoyager **81.4%** (text-only SOTA)
- 웹·GUI 자동화에 강함

CHAPTER 4 / 6

04_{/6}

MULTI-AGENT & A2A · ACP

이 챕터에서 답할 질문

- Plan-and-Execute · Reflexion · Multi-Agent · Graph-based 패턴은 언제?
- 단일 Agent에서 **Multi-Agent**로 넘어가는 시점
- **A2A** 프로토콜의 핵심 (Agent Card · Task lifecycle)
- **ACP**가 A2A로 통합된 배경 · 3-Layer 스택

— OTHER PATTERNS

그 외 주요 **AGENT** 패턴

01**PLAN-AND-EXECUTE**

Plan 단계와 Execute 단계 분리. 다 단계 리서치.

02**REFLEXION**

결과 자기평가 후 개선. 고난도 코드 생성.

03**MULTI-AGENT
SUPERVISOR**

전문가 Agent들을 Supervisor가 라우팅. CrewAI 방식.

04**GRAPH-BASED**

상태 기계로 명시적 전이.
LangGraph 철학.

오늘 다룰 패턴: Graph-based · LangGraph.

4 AGENT 패턴의 실전 유스케이스

REACT 사내 지식 QA

- Q: "지난 분기 매출 1위 제품?"
- Thought → `query_sales_db(Q3)` → Obs
- Final: "제품 A · \$3.2M"
- 짧은 루프, 도구 1~3번에 최적

PLAN-AND-EXECUTE 시장 조사 리포트

- Plan: [경쟁사·비교·차트·요약]
- Execute: 각 단계 순차 수행
- 20분 · 10페이지 리포트
- 길고 단계 명확한 작업

REFLEXION 고난도 코드 생성

- 1차 답안 → 테스트 3/1 실패
- 자가 개선 → 2차 답안 → 통과
- HumanEval +20%p 향상 사례
- 검증 가능한 영역(코드·수학)

MULTI-AGENT SUPERVISOR 블로그 팀

- Supervisor → Researcher (자료)
- → Writer (초안) → Editor (교정)
- 역할 분리 = 프롬프트 단순·품질 ↑
- CrewAI 기본 철학

단순 조회 ReAct · 장기 Plan-Execute · 품질 Reflexion · 역할 분리 Multi-Agent.

— WHEN TO SPLIT

언제 **MULTI-AGENT**로 넘어가는가

단일 AGENT로 충분

- 도구 5~10개, 한 도메인 내 작업
- 한 LLM이 도구·정책 머리에 다 담음
- 예: 사내 지식 QA, 고객 문의 1차

MULTI-AGENT 신호

- Tool 30개+ → 프롬프트 폭증
- 역할 분리 자연스러움 (Researcher · Writer · Editor)
- 도메인별 다른 모델·정책
- 부서 간 책임 경계 (다른 팀 Agent 협업)

합정 3가지 N제공 비용 가능 · Trace가 Agent 경계 가로질러야 (OTel) · Hop Count 상한으로 호출 루프(A→B→A) 방지.

— AGENT2AGENT PROTOCOL

A2A · OPAQUE AGENT들의 공통 규약

Google · Linux Foundation · 2025-04 · 2026-04 기준 150+ 조직 프로덕션

01

AGENT CARD

JSON 카드로 capabilities · auth · endpoint 노출. Agent를 찾을 수 있는 단위로 만드는 표준.

02

TASK LIFECYCLE

id · state · artifacts · history.
submitted → working → input-required → completed

03

TRANSPORT

JSON-RPC 2.0 over HTTP(S)
동기 · SSE 스트리밍 · 비동기 Push

04

OPACITY

내부 메모리·프롬프트·툴 미공개
블랙박스를 그대로 두고 협업
다른 벤더·프레임워크 Agent와 통신 가능

MCP는 단일 Agent의 도구 표준 · A2A는 Agent 간 협업 표준 — 다른 총입니다.

3-LAYER 프로토콜 스택

ACP (IBM · 2025) → A2A로 통합 (2026) HTTP-native · REST 기반 Agent 통신 프로토콜.
LF 산하에서 A2A에 합쳐져 Agent 간 통신 표준은 A2A 하나로 일원화됐습니다.

LAYER 3 WebMCP	Agent ↔ 웹 환경 · 브라우저·웹 콘텐츠 표준화 2026 들어 떠오른 신생 영역, 컨센서스 후보
LAYER 2 A2A	Agent ↔ Agent · Task · Card · JSON-RPC ← ACP merged, 150+ 조직 프로덕션
LAYER 1 MCP	Agent ↔ Tool · Resource · 단일 Agent의 외부 통합 2024-11 공개, 9,700만+ SDK 다운로드/월

세 층 모두 Agentic AI Foundation(Linux Foundation 산하, 146 회원사) 거버넌스 — 한 회사 표준이 아닌 산업 표준.

CHAPTER 5 / 6

05 / 6

LANGGRAPH로 구현

이 챕터에서 답할 질문

- LangChain과 LangGraph는 어떤 **역할 차이**를 갖는가?
- LangGraph **5 요소** (State · Node · Edge · ToolNode · Checkpointer)
- 기본 Tool-use Agent **루프 구조**
- State · Tool · 관측성 **설계 체크리스트**

— SAME COMPANY, DIFFERENT LAYER

LANGCHAIN VS LANGGRAPH

LANGCHAIN 빌딩 블록

- LLM 래퍼 · Retriever · Tool
- 선형 *Chain* 합성에 강함
- 짧은 Chain이면 충분

LANGGRAPH 실행 엔진

- 상태 그래프 (StateGraph)
- 체크포인팅 · HITL · 지속성 1급
- 프로덕션 거버넌스 필요시

선택 조건: 다단계 + 재시작 + 사람 승인 + 감사 로그 필요 시 LangGraph.

— LANGGRAPH = AGENT + WORKFLOW

AGENT VS WORKFLOW · 둘 다 LANGGRAPH로 구현

	WORKFLOW	AGENT
경로 결정	개발자가 사전 정의 (deterministic)	LLM이 매 step 동적 결정
예시	Routing · Parallelization · Orchestrator-Workers · Evaluator-Optimizer	ReAct 루프 · Plan-and-Act
예측 가능성	높음 · 테스트 쉬움	낮음 · trace 분석 필수
비용·지연	낮고 안정	높고 변동
적합 작업	도메인 좁음 · SLA·감사 중요	탐색·문제 다양
LangGraph 표현	Edge · Conditional Edge로 명시 흐름	ToolNode + LLM 루프 + Conditional Edge

LangGraph는 Agent 전용이 아님 · 같은 StateGraph 위에서 Workflow와 Agent를 자유롭게 섞을 수 있는 게 차별점.

→ Anthropic · "Building Effective Agents" (2024-12) · LangGraph docs

— CORE CONCEPTS

LANGGRAPH 5 요소

01**STATE**

TypedDict. 노드 간 공유
컨텍스트

02**NODE**

함수/Runnable. State →
업데이트

03**EDGE**

Conditional — 상태 기반
전이

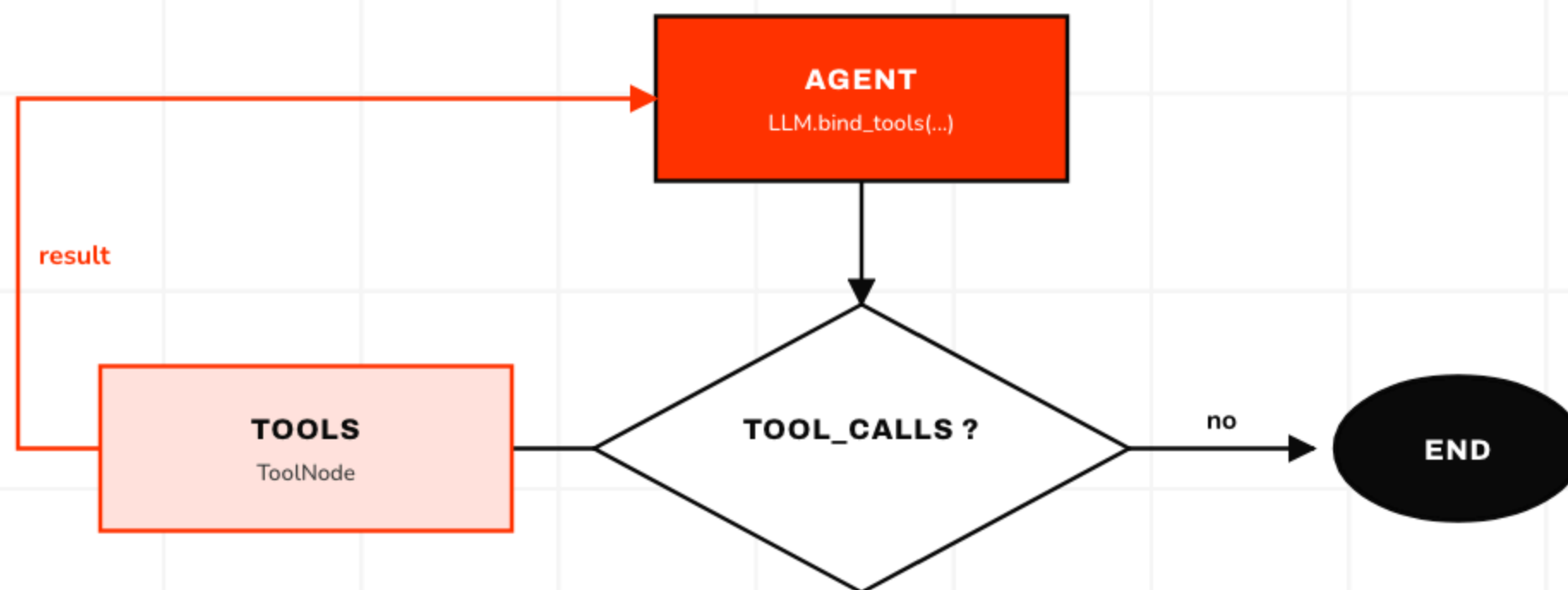
04**TOOLNODE**

prebuilt — 툴 실행 전용
노드

05**CHECKPOINTER**

SQLite/Redis → 재개 가능

기본 TOOL-USE 루프



두 노드 루프 = Agent의 심장. 모든 Agent의 최소 구조.

— STATE DESIGN

STATE는 직렬화 가능 필수

```
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]    # 대화 이력
    user_id: str
    tool_budget: int                          # 남은 tool 호출 횟수 · 무한루프 방지
    needs_approval: bool                     # 승인 필요 플래그
```

— TOOL DESIGN

TOOL 설계 6 체크리스트

- Docstring이 명확한가 (LLM이 읽는 유일한 설명)
- 인자 스키마가 엄격한가 (Pydantic · JSON Schema)
- Idempotent한가 (재시도 안전)
- 타임아웃이 있는가
- 권한 체크(user_id 기반)가 있는가
- 결과 크기 제한이 있는가 (LLM 컨텍스트 보호)

흔한 함정 제한 없이 도구가 10MB JSON 반환 → 다음 LLM 호출 컨텍스트 붕괴.

— OBSERVABILITY

관측성 설계

필수 항목

- 노드 진입/종료마다 OTel span
- Tool 호출 결과 요약 → trace attr
- 실패 메시지는 State에 기록

도구

- Langfuse · LangSmith
- OpenTelemetry 표준

패턴 툴 실패는 예외 throw 금지. 에러를 `state.messages`에 기록 → 모델이 스스로 복구.

CHAPTER 6 / 6

06 / 6

프로덕션 체크리스트

이 챕터에서 답할 질문

- Agent를 프로덕션에 올리기 전 7 체크리스트

— PRODUCTION READINESS

AGENT 프로덕션 7 체크리스트

- 01 Step Budget — 최대 루프 · 최대 토큰
- 02 Tool Timeout & Retry — 역등 키
- 03 Observability — trace · token/cost per trace
- 04 Human-in-the-loop — interrupt + resume
- 05 Policy — Tool 권한 · PII 스캔
- 06 Evals — 기대 경로 / 기대 도구 호출
- 07 Rollback — 프롬프트 · 모델 버전 되돌리기

Eval은 응답 문자열이 아닌 **경로·도구 호출**까지 검증해야 Regression 포착 가능.

CLOSING · COURSE WRAP-UP

END

코스 마무리

세 섹션 통합 한 줄

- RAG로 지식을 주입하고
- GATEWAY로 모델을 운영하고
- AGENT로 루프를 돌린다

— COURSE WRAP-UP

세 섹션의 한 줄 요약

RAG로 지식을 주입하고,
GATEWAY로 모델을 운영하고,
AGENT로 루프를 운영

☐ Agent 공식
(Loop+Tools+State+Policy)

☐ ReAct → ReWOO → Plan-and-Act 진화

☐ MCP · A2A · 3-Layer 프로토콜
스택

☐ 단일 Agent vs Multi-Agent 분리
기준

☐ LangChain vs LangGraph 레이어

☐ StateGraph + Checkpointer 최소
구조

☐ 프로덕션 7 체크리스트

THANK YOU.